Deep Learning Final Report: Escher Tiling Generation

Chengwei Liang, Jiarui Zhang, Xiawei Hu, Yunfan Xiao
Institute for Interdisciplinary Information Sciences, Tsinghua University
{lcw24,jiarui-z23,huxw23,xiaoyf23}@mails.tsinghua.edu.cn

Abstract

This report introduces two methods for synthesizing Escher-inspired tilings using generative techniques. Building on Aigerman and Groueix's framework for tileable mesh generation, we propose two approaches for multi-prompt generation: texture division and identical shape generation. These methods address challenges in boundary competition and enable the creation of diverse, aesthetically coherent tilings. Experimental results demonstrate the effectiveness of our approaches in producing intricate patterns that seamlessly blend artistic creativity with mathematical precision. Our code can be seen at https://git.tsinghua.edu.cn/jiarui-z23/generativeeschermeshes *in different branches.*

Acknowledgements

We extend our gratitude to the authors of [1] for their foundational contributions and to the open-source community for providing essential tools and resources. Their code work can bee seen at https://github.com/ThibaultGROUEIX/GenerativeEscherMeshes.

1. Introduction

Escher's tiling style is characterized by the seamless tessellation of the plane using interlocking, often biomorphic, shapes that exhibit rich symmetries and metamorphic transformations. In his seminal works of the 1930s-1950s, M. C. Escher explored how irregular, figural motifs (birds, fish, lizards) could fill the plane without gaps or overlaps, leveraging the 17 wallpaper groups of planar symmetry to achieve both mathematical rigor and visual delight [3]. These patterns challenge the viewer's perception by combining local complexity with global periodicity, and they remain a cornerstone example of art informed by mathematical structure.

Early computational efforts to replicate Escher-style tilings focused on algorithmic "Escherization," wherein a user-defined motif is deformed to satisfy edge-matching constraints. Kaplan and Salesin (2000) introduced one of the first fully automatic pipelines, optimizing motif boundaries to fit symmetry criteria while preserving recognizability [4]. However, such methods often rely on handcrafted shape representations and explicit optimization, limiting their ability to generalize to highly intricate or stylistically novel patterns.

Recent advances in deep generative modeling open new avenues for synthesizing Escher-inspired artworks. In particular, Aigerman and Groueix (2024) propose a mesh-based neural framework that learns to warp and interlock cellular patterns into continuous, Escher-esque tilings, producing high-fidelity, aesthetically coherent meshes without manual tuning [1]. By combining implicit shape representations with adversarial training, their approach demonstrates how learned priors over geometry and texture can capture the spirit of Escher's art at scale—laying the groundwork for further exploration of style-driven, generative tiling systems.

2. Related Work

Early efforts in automating Escher-style tilings focused on deforming input motifs to satisfy planar symmetry constraints while preserving visual recognizability. Kaplan and Salesin pioneered this "Escherization" pipeline, formulating a boundary-curve optimization that fits a user-defined shape into one of the 17 planar symmetry groups (wallpaper groups) with minimal distortion [4]. Building on this, Sugihara developed an interactive system that blends template-based tiling with smooth morphing and figure-ground reversals, enabling artists to explore continuous metamorphoses reminiscent of Escher's *Sky and Water I* [8]. While powerful, these methods rely on handcrafted representations and global optimizations, limiting their flexibility for intricate or unconstrained pattern styles.

To expand the space of Escher-like designs, researchers have extended classical techniques in two dimensions. Nagata and Imahori proposed a method that evaluates similarity using as-rigid-as-possible deformation energy during optimization to generate two distinct shapes (dihedral Escher-like tilings). This approach allows two distinct motif

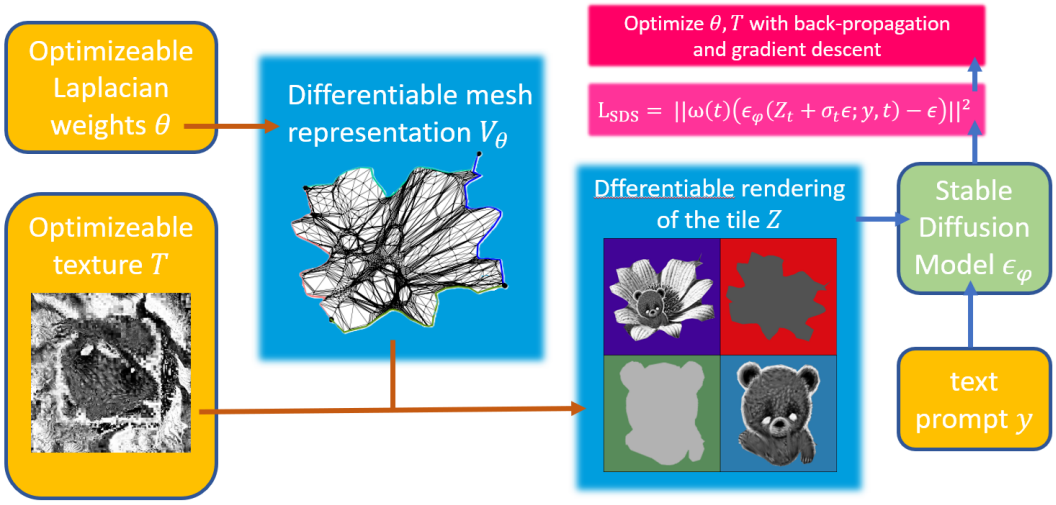


Figure 1. Overview of the method.

classes to interlock under user-controllable arrangements, providing richer combinatorial possibilities than single-shape tilings [5]. Ouyang et al. explored conformal mapping to create spiral tessellations with cyclic symmetry, introducing decorative, Escher-inspired spirals that transcend conventional wallpaper groups [6]. These advances demonstrate how geometric flexibility and user control can yield novel aesthetic effects, though they remain bound to explicit deformation frameworks.

Expanding into three dimensions, Howison adapted Escherization concepts for volumetric design. His CAD tools enable the creation of space-filling 3D tiles by generalizing planar edge-matching to solid interlocking parts, opening the door to "Escher tiles."

3. Aigerman and Groueix's Method

Our approach builds on Aigerman and Groueix's 2024 work, "Generative Escher Meshes" [1]. Figure 1 provides an overview of their method. The framework employs a differentiable representation to model a tile, enabling optimization to better match text prompts while maintaining tileability. The optimization process leverages a loss function L_{SDS}, defined by Score Distillation Sampling (SDS) [7], which provides gradient signals to the differentiable representation.

Differentiable Representation. The tile representation consists of two components: mesh and texture. The mesh representation θ is an unconstrained vector in $\mathbb{R}^k$, mapped to a tileable 2D triangular mesh V_θ. The texture representation T is a 2D image, where each vertex in the mesh corresponds to fixed UV coordinates in the image. Consequently, the tile Z is rendered from the representation (θ, T).

The mesh representation satisfies three key properties: validity, completeness, and differentiability. Validity ensures that any $\theta \in \mathbb{R}^k$ maps to a tileable, non-overlapping mesh V_θ. Completeness guarantees that any valid mesh corresponds to some θ. Differentiability ensures that the mapping from θ to V_θ is differentiable, enabling gradient computation. The mesh V_θ is obtained by solving a linear system defined by:

- Constraints ensuring tileability, derived from the 17 types of tileable meshes [2].
- Lagrangian equations ensuring non-overlapping meshes:

$$\sum_{v_j \in N^i} L^\theta_{ij}(v_i - v_j) = 0, \tag{1}$$

where N^i are neighbors of v_i, and

$$L^\theta_{ij} = \begin{cases} -f(\theta_{ij}) & \text{if edge } (v_i, v_j) \text{ exists,} \\ \sum_k f(\theta_{ik}) & i = j, \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

Text-guided Optimization. The optimization process aims to minimize the diffusion training loss, defined as:

$$L_{\text{Diff}}(Z) = \mathbb{E}_{t,\epsilon}\left[w'(t)\|\hat{\epsilon}_\phi(z_t; y, t) - \epsilon\|^2\right], \tag{3}$$

where $z_t = \alpha_t Z + \sigma_t \epsilon$ represents the noisy version of the tile Z at timestep t, with noise ϵ, and $w'(t)$ is a weighting function dependent on the timestep. The gradient of the loss with respect to the tile Z is given by:

$$\nabla_Z L_{\text{Diff}}(Z) = \mathbb{E}_{t,\epsilon}\left[w(t)\left(\hat{\epsilon}_\phi(z_t; y, t) - \epsilon\right)\frac{\partial \hat{\epsilon}_\phi(z_t; y, t)}{\partial z_t}\right], \tag{4}$$

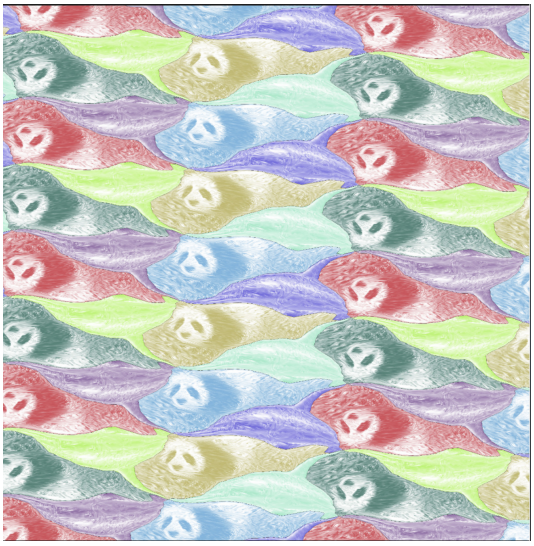


Figure 2. “A panda” + “A dolphin”

where $w(t)$ is a modified weighting function that incorporates $\alpha_t \mathbf{I} = \partial z_t / \partial Z$. Interestingly, it has been observed that omitting the term $\dfrac{\partial \hat{\epsilon}_\phi(z_t; y, t)}{\partial z_t}$ not only reduces computational cost but also improves performance. Consequently, the gradient of the simplified loss, denoted as L_{SDS}, is expressed as:

$$\nabla_Z L_{\text{SDS}}(Z) \triangleq \mathbb{E}_{t,\epsilon}\left[w(t)\left(\hat{\epsilon}_\phi(z_t; y, t) - \epsilon\right)\right]. \qquad (5)$$

Multi-prompt Generation. Aigerman and Groueix extend their method to multi-prompt generation, enabling multiple contents to appear in the result. For p prompts, the tile is split into p sub-tiles, each assigned a prompt. However, this approach suffers from loss competition among sub-tiles. We propose two novel approaches to address this issue. We emphasize that our approaches do not increase computational costs compared to the original method.

4. Approach I: Texture Division

In this approach, we aim to divide the texture in a more flexible manner. For given prompts $y_1, y_2, \ldots, y_p$, we manually designate specific areas of the texture to correspond to each prompt. The SDS loss is computed for each prompt and its associated texture area, and the total loss is obtained by summing these individual SDS losses.

Figure 3 illustrates an example of a division referred to as flower mode. Using this UV-texture division, two images are rendered and processed by the stable diffusion model to compute the SDS loss gradients, which are subsequently aggregated.

The flower mode offers greater flexibility compared to the original approach. Specifically, the two prompts do not need to share a common boundary, and the prompt assigned to the inner region is not constrained by tileability requirements. Additional results for flower mode are presented in Figures 5a and Figures 5b.

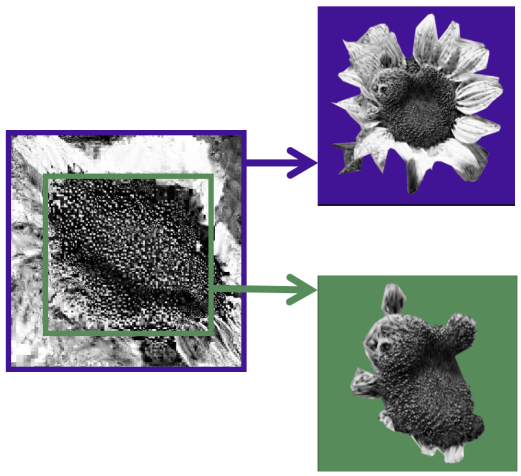


Figure 3. UV-texture division in flower mode.

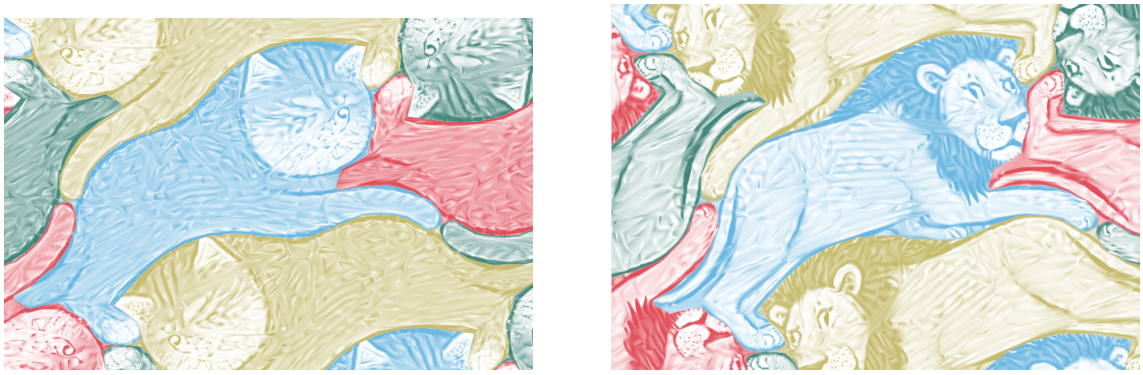
We also implemented another mode, referred to as share-body mode. In this mode, the texture is divided into three parts: the upper part, the middle part, and the lower part. The upper and middle parts are assigned to the first prompt, while the middle and lower parts are assigned to the second prompt. However, since both prompts influence the tile boundary, the performance of this mode is inferior to that of the flower mode. An example of share-body mode is shown in Figure 5c.

5. Approach II: Multi-prompt Generation with Identical Shape

Instead of dividing a tile into sub-tiles, this approach generates p tiles with identical shapes. For instance, in Figure 4, we generate a tile of a cat and a tile of a lion who share the same shape. This allows us to create a picture of tiling that incorporates both a cat and a lion. This method simplifies the prompt selection by requiring only a subset of prompts with similar shapes, as opposed to the original approach, which necessitated selecting prompts that could be seamlessly stitched together inside a tile. A simpler alternative is to use p identical prompts so that they are guaranteed to have similar shapes.

To accommodate multiple tiles, we extend the tile representation. This can be achieved in two ways:

- **Single Mesh Representation**. The mesh representation θ remains a single vector in $\mathbb{R}^k$, while the texture representation T is extended to $T_1, T_2, \ldots, T_p$. During the pipeline, p tiles $Z_1, Z_2, \ldots, Z_p$ are rendered from $(V_\theta, T_1), \cdots, (V_\theta, T_p)$, respectively. The SDS loss is



(a) “A cat” (b) “A lion”

Figure 4. Two tiles with identical shapes and different contents.

computed for each tile-prompt pair (Z_i, y_i), and the total loss is the sum of these individual losses.

- **Multiple Mesh Representations**. Enforcing a single mesh for all tiles may lead to competition between different prompts and suboptimal results. To address this, the mesh representation is extended to $\theta_1, \theta_2, \ldots, \theta_p$, ensuring that the meshes $V_{\theta_1}, V_{\theta_2}, \ldots, V_{\theta_p}$ share the same coordinates for boundary vertices (but not for internal vertices). This allows $V_{\theta_1}, V_{\theta_2}, \ldots, V_{\theta_p}$ to be jointly solved in a linear system with additional constraints to align the boundary vertices. The tiles $Z_1, Z_2, \ldots, Z_p$ are then rendered from $(V_{\theta_1}, T_1), (V_{\theta_2}, T_2), \ldots, (V_{\theta_p}, T_p)$, and the loss is computed similarly to the single mesh representation.

We conducted experiments to compare the two methods of representation extension. The results suggest that both single mesh and multiple mesh representations achieve comparable performance. This can be attributed to the high resolution of the texture, which is capable of adapting to any natural mesh. However, the multiple mesh representation introduces a significant computational overhead, as the number of variables in the linear system increases linearly with the number of prompts, leading to a time complexity that grows at least quadratically. For a scenario involving two prompts, the single mesh representation requires approximately 1 hour, whereas the multiple mesh representation takes around 3.5 hours.

The results of Approach II are shown in Figure 6. The first row illustrates results for multiple different prompts. For a given set of prompts, our method identifies a shape suitable for all prompts and generates corresponding textures for each. The second row demonstrates results for multiple identical prompts. This approach naturally extends single-prompt generation to multi-prompt generation, enabling more diverse tiling than before.

6. Combination of Approach I and Approach II

By combining Approach I and Approach II, we can generate diverse tiles, each containing multiple elements. The outcomes are illustrated in Figure 7. It is important to note that this method relies on multiple mesh representations described in Approach II, which leads to an increase in generation time.

7. Conclusion

In this work, we extended the generative Escher tiling framework by introducing two novel approaches for multi-prompt generation: texture division and identical shape generation. These methods avoid the challenges posed by boundary competition, enabling the creation of diverse, multi-content tiles. Our experiments demonstrate that these approaches not only preserve tileability but also enhance the aesthetic and functional diversity of the generated patterns. Future research could explore integrating additional constraints, such as user-defined styles or physical properties, to expand the applicability of these methods.

References

[1] Noam Aigerman and Thibault Groueix. Generative escher meshes. In *ACM SIGGRAPH 2024 Conference Papers*, pages 1–11, 2024. 1, 2

[2] J.H. Conway, H. Burgiel, and C. Goodman-Strauss. *The Symmetries of Things*. Taylor & Francis, 2008. 2

[3] Maurits Cornelis Escher. *The Magic of M. C. Escher*. Harry N. Abrams, 2000. 1

[4] Craig S Kaplan and David H Salesin. Escherization. In *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, pages 499–510, 2000. 1

[5] Yuichi Nagata and Shinji Imahori. Creation of dihedral escher-like tilings based on as-rigid-as-possible deformation. *ACM Transactions on Graphics*, 43(2):1–18, 2024. 2

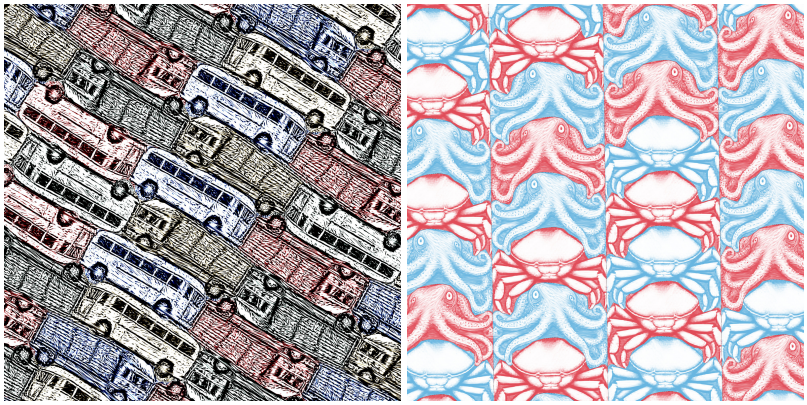
[6] Peichang Ouyang, Kwok Wai Chung, David Bailey, Alain Nicolas, and Krzysztof Gdawiec. Generation of advanced escher-like spiral tessellations. *The Visual Computer*, 38(11): 3923–3935, 2022. 2

[7] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion, 2022. 2

[8] Kokichi Sugihara. Computer-aided generation of escher-like sky and water tiling patterns. *Journal of Mathematics and the Arts*, 3(4):195–207, 2009. 1

(a) “A flower” + “A Winnie the Pooh” (b) “A flower” + “A bear” (c) “A dolphin” + “A dolphin”

Figure 5. Results of Approach I. The first two pictures show results for flower mode. The third subfigure shows picture for share-body mode.



“A bus” + “A truck” “A crab” + “An octopus”

“A flower” + “A bunch of grapes” “An alligator” + “A wolf”

“A flower” x2 + “A butterfly” x2 “A bear” x2

“A bear” x4 “A monkey” x4

Figure 6. Results for Approach II. The first 4 pictures show results for different prompts, while the last 4 pictures show results for identical prompts.

”A dolphin” + ”An oyster”, ”An oyster” + ”A dolphin”

”A flower” + ”An apple”, ”A flower” + ”An orange”

Figure 7. Results of combining Approach I and Approach II.